NATIONAL UNIVERSITY
COLLEGE OF COMPUTING AND
INFORMATION TECHNOLOGIES

COMPILATION OF FINAL PROJECT

Course Code/Subject	CCOPSYSL	**Term/AY**	**1st / 25-26**
Section	COM232	**Date**	October 14, 2025
Group # | Leader Name	Talosig, Jay Arre P.		
CCOPSYSL GitHub	https://github.com/flexycode/CCOPSYSL/tree/main/x-Final%20Project		

FINAL PROJECT
Deadlock Detection and Prevention

Prepared by:

Talosig, Jay Arre P.

_____

Signature over group leader name

Project Title: Deadlock Detection and Prevention System

1. Introduction

Project Overview: This project implements a comprehensive simulation of deadlock detection and prevention algorithms in a single Java file (Groupproj.java). The system demonstrates how operating systems handle situations where processes get stuck waiting for resources in a cyclic manner, potentially leading to system deadlocks.

Motivation: Deadlocks are a critical issue in operating systems that can bring entire systems to a halt. This single-file implementation provides a complete, self-contained educational tool that demonstrates practical implementation of deadlock handling algorithms without complex project setup.

2. Objectives

- To implement Banker's Algorithm for deadlock detection and avoidance in a single Java class
- To create a Resource Allocation Graph system for cycle detection within one file
- To simulate various deadlock prevention techniques through an interactive menu
- To develop a user-friendly system that demonstrates deadlock scenarios without multiple file dependencies
- To provide a complete educational tool in one compilable Java file

3. Prerequisites

Technical Requirements:

- Java Development Kit (JDK) 8 or higher
- Command-line compiler or any Java IDE
- Single file: Groupproj.java

Knowledge Requirements:

- Understanding of process synchronization concepts
- Knowledge of resource allocation principles
- Familiarity with deadlock conditions
- Basic Java programming knowledge

4. Project Description

Methodology:
The project uses a single Java class with nested static classes:

- **Process**: Models system processes with resource allocation needs
- **BankersAlgorithm**: Implements the Banker's Algorithm for safety checks
- **ResourceAllocationGraph**: Handles graph-based deadlock detection
- **Main class**: Provides user interface and simulation coordination

Single-File Architecture:

java
```
public class Groupproj {
    static class Process { ... }
    static class BankersAlgorithm { ... }
```

```
    static class ResourceAllocationGraph { ... }
    // Main method and interactive simulation methods
}
```

Key Features:

- Complete implementation in one Java file
- Interactive menu-driven interface
- Real-time deadlock detection using multiple algorithms
- No external dependencies or multiple file compilation
- Input validation and error handling

5. Implementation

Single File Structure:
The entire project is contained in Groupproj.java with:

- 4 nested static classes
- Interactive input methods
- 4 main simulation scenarios
- Comprehensive error handling

Core Algorithms in One File:

1. Banker's Safety Algorithm:

   ```java
   static class BankersAlgorithm {
       public boolean isSafeState() {
           // Complete safety check implementation
       }
   }
   ```

2. Resource Allocation Graph:

   ```java
   static class ResourceAllocationGraph {
       public List<Integer> detectDeadlock() {
           // DFS-based cycle detection
       }
   }
   ```

Setup Instructions:

1. Save the code as Groupproj.java
2. Compile: javac Groupproj.java
3. Run: java Groupproj
4. Follow the interactive menu prompts

No Additional Dependencies Required - everything is in one file!

6. Testing

Testing Strategy:
- All testing done within the single file implementation
- Menu option testing for all features
- Input validation testing
- Algorithm accuracy verification

Test Results:
- ✅ **Option 1**: Banker's Algorithm - Safe state detection working
- ✅ **Option 2**: Resource request simulation functioning correctly
- ✅ **Option 3**: Resource allocation graph deadlock detection accurate
- ✅ **Option 4**: Prevention techniques properly demonstrated
- ✅ Input validation working for all scenarios
- ✅ Error handling robust for invalid inputs

7. Challenges Faced

Single-File Design Challenges:
- Organizing multiple algorithms within one class structure
- Maintaining code readability without package separation
- Managing nested class dependencies

Solutions Implemented:
- Used clear static nested class structure
- Comprehensive commenting for organization
- Logical method separation within single file

8. Conclusion

Achievements:
- Successfully implemented multiple deadlock algorithms in one Java file
- Created a complete educational tool without file dependencies
- Developed robust, interactive system in single compilation unit
- Demonstrated complex OS concepts in accessible format

Single-File Advantages:
- Easy submission and grading
- Simple compilation process
- No project configuration needed
- Portable across different systems

9. References

1. Operating System Concepts by *Silberschatz, Galvin, and Gagne*
2. Banker's Algorithm academic resources
3. Java Documentation for single-class implementation

10. Appendix

File Structure:

```
Groupproj.java (Single File)
├── static class Process
├── static class BankersAlgorithm
├── static class ResourceAllocationGraph
├── main() method
└── Simulation methods:
    ├── simulateBankersSafeState()
    ├── simulateBankersResourceRequest()
    ├── simulateResourceAllocationGraph()
    └── simulateDeadlockPrevention()
```

Compilation and Execution:

```
# Compile
javac Groupproj.java

# Run
java Groupproj
```

Recommended Test Inputs :

- Option 1: 5 processes, 3 resources, available: 3,3,2
- Option 2: 3 processes, 2 resources, request from P1: 1,0
- Option 3: 3 processes, 3 resources with circular dependencies

Complete Code Features in single file:

- ✅ All four deadlock detection scenarios
- ✅ Interactive menu system
- ✅ Input validation
- ✅ Error handling
- ✅ Educational demonstrations
- ✅ No external dependencies

Complete Source Code down below:

SOURCE CODE

```
import java.util.*;

public class Groupproj {

    // ===PROCESS CLASS ===
    static class Process {
        private int id;
        private int[] allocated;
        private int[] maximum;
        private int[] need;
        private boolean finished;

        public Process(int id, int[] allocated, int[]
maximum) {
            this.id = id;
            this.allocated = allocated.clone();
            this.maximum = maximum.clone();
            this.need = new int[allocated.length];
            this.finished = false;

            // Calculate need matrix
            for (int i = 0; i < allocated.length; i++) {
                need[i] = maximum[i] - allocated[i];
            }
        }

        public int getId() { return id; }
        public int[] getAllocated() { return
allocated; }
        public int[] getMaximum() { return
maximum; }
        public int[] getNeed() { return need; }
        public boolean isFinished() { return
finished; }
        public void setFinished(boolean finished)
{ this.finished = finished; }

        public boolean canBeSatisfied(int[]
available) {
            for (int i = 0; i < need.length; i++) {
                if (need[i] > available[i]) {
                    return false;
                }
            }
            return true;
        }

        public void releaseResources(int[]
available) {
            for (int i = 0; i < allocated.length; i++) {
                available[i] += allocated[i];
                allocated[i] = 0;
            }
            finished = true;
        }

        @Override
        public String toString() {
            return "P" + id;
        }
    }

    // === BANKER'S ALGORITHM ===
    static class BankersAlgorithm {
        private Process[] processes;
        private int[] available;
        private int resourceCount;

        public BankersAlgorithm(Process[]
processes, int[] available) {
            this.processes = processes;
            this.available = available.clone();
            this.resourceCount = available.length;
        }

        /**
         * Check if the system is in a safe state
using Banker's Algorithm
         */
        public boolean isSafeState() {
            int[] work = available.clone();
            boolean[] finish = new
boolean[processes.length];
            List<Process> safeSequence = new
ArrayList<>();

            // Initialize finish array
            for (int i = 0; i < processes.length; i++)
{
                finish[i] = processes[i].isFinished();
            }

            boolean found;
            do {
                found = false;
                for (int i = 0; i < processes.length;
i++) {
                    if (!finish[i] &&
processes[i].canBeSatisfied(work)) {
```

SOURCE CODE

```
                // Process can be satisfied - simulate execution
                processes[i].releaseResources(work);
                finish[i] = true;
                safeSequence.add(processes[i]);
                found = true;
                System.out.println("  Executing " + processes[i] + ", Work: " + Arrays.toString(work));
            }
        }
    } while (found);

    // Check if all processes finished
    boolean allFinished = true;
    for (boolean f : finish) {
        if (!f) {
            allFinished = false;
            break;
        }
    }

    if (allFinished) {
        System.out.println("  ✓ System is in SAFE state");
        System.out.print("  Safe sequence: ");
        for (int i = 0; i < safeSequence.size(); i++) {
            System.out.print(safeSequence.get(i));
            if (i < safeSequence.size() - 1) System.out.print(" → ");
        }
        System.out.println();
    } else {
        System.out.println("  ✗ System is in UNSAFE state - DEADLOCK DETECTED!");
        System.out.print("  Deadlocked processes: ");
        for (int i = 0; i < processes.length; i++) {
            if (!finish[i]) {
                System.out.print("P" + processes[i].getId() + " ");
            }
        }
        System.out.println();
    }

    return allFinished;
}

/**
 * Simulate resource request from a process
 */
public boolean requestResources(int processId, int[] request) {
    Process process = processes[processId];
    int[] allocated = process.getAllocated();
    int[] need = process.getNeed();

    System.out.println("\n  === Resource Request Simulation ===");
    System.out.println("  " + process + " requests: " + Arrays.toString(request));

    // Step 1: Check if request <= need
    for (int i = 0; i < resourceCount; i++) {
        if (request[i] > need[i]) {
            System.out.println("  Error: Request exceeds maximum claim");
            return false;
        }
    }

    // Step 2: Check if request <= available
    for (int i = 0; i < resourceCount; i++) {
        if (request[i] > available[i]) {
            System.out.println("  Process must wait - resources not available");
            return false;
        }
    }

    // Step 3: Try to allocate resources temporarily
    for (int i = 0; i < resourceCount; i++) {
        available[i] -= request[i];
        allocated[i] += request[i];
        need[i] -= request[i];
    }
```

SOURCE CODE

```
        // Step 4: Check if system remains in
safe state
        System.out.println("  Checking if
system remains safe after allocation...");
        boolean safe = isSafeState();

        if (!safe) {
            // Rollback allocation
            System.out.println("  Rolling back
allocation - request would cause deadlock");
            for (int i = 0; i < resourceCount; i++)
{
                available[i] += request[i];
                allocated[i] -= request[i];
                need[i] += request[i];
            }
        } else {
            System.out.println("  Request
granted safely");
        }

        return safe;
    }
  }

  // === RESOURCE ALLOCATION GRAPH
===
  static class ResourceAllocationGraph {
    private Map<Integer, List<Integer>>
processToResource;
    private Map<Integer, List<Integer>>
resourceToProcess;
    private Map<Integer, Integer>
resourceInstances;

    public ResourceAllocationGraph() {
        processToResource = new
HashMap<>();
        resourceToProcess = new
HashMap<>();
        resourceInstances = new
HashMap<>();
    }

    public void addProcess(int processId) {
        processToResource.put(processId,
new ArrayList<>());
    }

    public void addResource(int resourceId,
int instances) {
        resourceToProcess.put(resourceId,
new ArrayList<>());
        resourceInstances.put(resourceId,
instances);
    }

    public void addRequestEdge(int
processId, int resourceId) {

processToResource.get(processId).add(reso
urceId);
    }

    public void addAssignmentEdge(int
resourceId, int processId) {

resourceToProcess.get(resourceId).add(proc
essId);
    }

    /**
     * Detect deadlock using cycle detection
     */
    public List<Integer> detectDeadlock() {
        Set<Integer> visited = new
HashSet<>();
        Set<Integer> recursionStack = new
HashSet<>();
        Map<Integer, Integer> parent = new
HashMap<>();
        List<Integer> deadlockProcesses =
new ArrayList<>();

        for (int processId :
processToResource.keySet()) {
            if (!visited.contains(processId)) {
                if (dfsCycleDetection(processId,
visited, recursionStack, parent)) {
                    int cycleStart =
findCycleStart(processId, parent);
                    reconstructCycle(cycleStart,
parent, deadlockProcesses);
                    break;
                }
            }
        }

        return deadlockProcesses;
```

SOURCE CODE

```
    }

    private boolean dfsCycleDetection(int current, Set<Integer> visited,
                                      Set<Integer> recursionStack, Map<Integer, Integer> parent) {
        visited.add(current);
        recursionStack.add(current);

        if (processToResource.containsKey(current)) {
            for (int resourceId : processToResource.get(current)) {
                if (resourceToProcess.containsKey(resourceId)) {
                    for (int nextProcess : resourceToProcess.get(resourceId)) {
                        if (!visited.contains(nextProcess)) {
                            parent.put(nextProcess, current);
                            if (dfsCycleDetection(nextProcess, visited, recursionStack, parent)) {
                                return true;
                            }
                        } else if (recursionStack.contains(nextProcess)) {
                            parent.put(nextProcess, current);
                            return true;
                        }
                    }
                }
            }
        }

        recursionStack.remove(current);
        return false;
    }

    private int findCycleStart(int node, Map<Integer, Integer> parent) {
        Set<Integer> seen = new HashSet<>();
        while (!seen.contains(node)) {
            seen.add(node);
            node = parent.get(node);
        }
        return node;
    }

    private void reconstructCycle(int start, Map<Integer, Integer> parent, List<Integer> cycle) {
        int current = start;
        do {
            cycle.add(current);
            current = parent.get(current);
        } while (current != start);
        cycle.add(start);
        Collections.reverse(cycle);
    }

    public void printGraph() {
        System.out.println("\n  === Resource Allocation Graph ===");
        System.out.println("  Request Edges (Process → Resource):");
        for (Map.Entry<Integer, List<Integer>> entry : processToResource.entrySet()) {
            for (int resourceId : entry.getValue()) {
                System.out.println("    P" + entry.getKey() + " → R" + resourceId);
            }
        }

        System.out.println("\n  Assignment Edges (Resource → Process):");
        for (Map.Entry<Integer, List<Integer>> entry : resourceToProcess.entrySet()) {
            for (int processId : entry.getValue()) {
                System.out.println("    R" + entry.getKey() + " → P" + processId);
            }
        }
    }

    // === INPUT UTILITIES ===
    static Scanner scanner = new Scanner(System.in);
```

SOURCE CODE

```
    private static int getIntInput(String prompt)
{
        System.out.print(prompt);
        while (!scanner.hasNextInt()) {
            System.out.println("Invalid input!
Please enter a number.");
            scanner.next();
            System.out.print(prompt);
        }
        return scanner.nextInt();
    }

    private static int[]
getResourceArrayInput(String
resourceName, int size) {
        int[] resources = new int[size];
        System.out.println("Enter " +
resourceName + " for " + size + "
resources:");
        for (int i = 0; i < size; i++) {
            resources[i] = getIntInput("  Resource
R" + i + ": ");
        }
        return resources;
    }

    private static int[]
getProcessResourceInput(int processId, int
resourceCount, String type) {
        int[] resources = new int[resourceCount];
        System.out.println("Enter " + type + " for
Process P" + processId + ":");
        for (int i = 0; i < resourceCount; i++) {
            resources[i] = getIntInput("  Resource
R" + i + ": ");
        }
        return resources;
    }

    // === MAIN SIMULATION ===
    public static void main(String[] args) {
        System.out.println("====");
        System.out.println("   DEADLOCK
DETECTION & PREVENTION");
        System.out.println("        SYSTEM
SIMULATION");
        System.out.println("====\n");

        showMainMenu();
        scanner.close();
    }

    private static void showMainMenu() {
        while (true) {
            System.out.println("\n=== MAIN
MENU ===");
            System.out.println("1. Banker's
Algorithm - Safe State Detection");
            System.out.println("2. Banker's
Algorithm - Resource Request");
            System.out.println("3. Resource
Allocation Graph - Deadlock Detection");
            System.out.println("4. Deadlock
Prevention Techniques");
            System.out.println("5. Exit");
            System.out.print("Choose an option
(1-5): ");

            int choice = getIntInput("");

            switch (choice) {
                case 1:
                    simulateBankersSafeState();
                    break;
                case 2:

simulateBankersResourceRequest();
                    break;
                case 3:

simulateResourceAllocationGraph();
                    break;
                case 4:
                    simulateDeadlockPrevention();
                    break;
                case 5:
                    System.out.println("Thank you
for using the Deadlock Detection System!");
                    return;
                default:
                    System.out.println("Invalid
choice! Please enter 1-5.");
            }
        }
    }

    private static void
simulateBankersSafeState() {
```

SOURCE CODE

```
        System.out.println("\n=== BANKER'S
ALGORITHM - SAFE STATE DETECTION
===");

        // Get number of processes and
resources
        int numProcesses = getIntInput("Enter
number of processes: ");
        int numResources = getIntInput("Enter
number of resource types: ");

        // Get available resources
        System.out.println("\n--- AVAILABLE
RESOURCES ---");
        int[] available =
getResourceArrayInput("available instances",
numResources);

        // Get process information
        Process[] processes = new
Process[numProcesses];
        System.out.println("\n--- PROCESS
INFORMATION ---");
        for (int i = 0; i < numProcesses; i++) {
            System.out.println("\nProcess P" + i +
":");
            int[] allocated =
getProcessResourceInput(i, numResources,
"allocated resources");
            int[] maximum =
getProcessResourceInput(i, numResources,
"maximum resources");
            processes[i] = new Process(i,
allocated, maximum);
        }

        // Run Banker's Algorithm
        System.out.println("\n--- RUNNING
BANKER'S ALGORITHM ---");
        BankersAlgorithm banker = new
BankersAlgorithm(processes, available);
        banker.isSafeState();
    }

    private static void
simulateBankersResourceRequest() {
        System.out.println("\n=== BANKER'S
ALGORITHM - RESOURCE REQUEST
===");

        // Get system configuration
        int numProcesses = getIntInput("Enter
number of processes: ");
        int numResources = getIntInput("Enter
number of resource types: ");

        // Get available resources
        System.out.println("\n--- AVAILABLE
RESOURCES ---");
        int[] available =
getResourceArrayInput("available instances",
numResources);

        // Get process information
        Process[] processes = new
Process[numProcesses];
        System.out.println("\n--- PROCESS
INFORMATION ---");
        for (int i = 0; i < numProcesses; i++) {
            System.out.println("\nProcess P" + i +
":");
            int[] allocated =
getProcessResourceInput(i, numResources,
"allocated resources");
            int[] maximum =
getProcessResourceInput(i, numResources,
"maximum resources");
            processes[i] = new Process(i,
allocated, maximum);
        }

        // Get resource request
        System.out.println("\n--- RESOURCE
REQUEST ---");
        int processId = getIntInput("Enter
process ID making request (0-" +
(numProcesses-1) + "): ");
        if (processId < 0 || processId >=
numProcesses) {
            System.out.println("Invalid process
ID!");
            return;
        }

        int[] request = new int[numResources];
        System.out.println("Enter resource
request for Process P" + processId + ":");
        for (int i = 0; i < numResources; i++) {
            request[i] = getIntInput("  Resource R"
+ i + ": ");
```

SOURCE CODE

```
        }

        // Process the request
        System.out.println("\n--- PROCESSING
REQUEST ---");
        BankersAlgorithm banker = new
BankersAlgorithm(processes, available);
        banker.requestResources(processId,
request);
    }

    private static void
simulateResourceAllocationGraph() {
        System.out.println("\n=== RESOURCE
ALLOCATION GRAPH ===");

        ResourceAllocationGraph rag = new
ResourceAllocationGraph();

        // Get number of processes and
resources
        int numProcesses = getIntInput("Enter
number of processes: ");
        int numResources = getIntInput("Enter
number of resources: ");

        // Add processes
        for (int i = 0; i < numProcesses; i++) {
            rag.addProcess(i);
        }

        // Add resources
        for (int i = 0; i < numResources; i++) {
            int instances = getIntInput("Enter
instances for Resource R" + i + ": ");
            rag.addResource(i, instances);
        }

        // Get request edges
        System.out.println("\n--- REQUEST
EDGES (Process → Resource) ---");
        System.out.println("Enter process-
resource request pairs (enter -1 to stop):");
        while (true) {
            int processId = getIntInput("  Process
ID (-1 to stop): ");
            if (processId == -1) break;
            if (processId < 0 || processId >=
numProcesses) {
                System.out.println("Invalid process
ID!");
                continue;
            }

            int resourceId = getIntInput("
Resource ID: ");
            if (resourceId < 0 || resourceId >=
numResources) {
                System.out.println("Invalid resource
ID!");
                continue;
            }

            rag.addRequestEdge(processId,
resourceId);
        }

        // Get assignment edges
        System.out.println("\n--- ASSIGNMENT
EDGES (Resource → Process) ---");
        System.out.println("Enter resource-
process assignment pairs (enter -1 to stop):");
        while (true) {
            int resourceId = getIntInput("
Resource ID (-1 to stop): ");
            if (resourceId == -1) break;
            if (resourceId < 0 || resourceId >=
numResources) {
                System.out.println("Invalid resource
ID!");
                continue;
            }

            int processId = getIntInput("  Process
ID: ");
            if (processId < 0 || processId >=
numProcesses) {
                System.out.println("Invalid process
ID!");
                continue;
            }

            rag.addAssignmentEdge(resourceId,
processId);
        }

        // Detect deadlock
        System.out.println("\n--- DETECTING
DEADLOCK ---");
```

SOURCE CODE

```
        rag.printGraph();

        List<Integer> deadlockProcesses =
rag.detectDeadlock();
        if (!deadlockProcesses.isEmpty()) {
            System.out.println("\n  ✗ DEADLOCK
DETECTED!");
            System.out.print("  Deadlock cycle: ");
            for (int i = 0; i <
deadlockProcesses.size(); i++) {
                System.out.print("P" +
deadlockProcesses.get(i));
                if (i < deadlockProcesses.size() - 1)
System.out.print(" → ");
            }
            System.out.println();
        } else {
            System.out.println("\n  ✓ No deadlock
detected");
        }
    }

    private static void
simulateDeadlockPrevention() {
        System.out.println("\n=== DEADLOCK
PREVENTION TECHNIQUES ===");

        System.out.println("\n1. MUTUAL
EXCLUSION BREAKING:");
        System.out.println("    - Use shared
resources or read-only access");
        System.out.println("    - Example: Multiple
processes can read a file simultaneously");

        System.out.println("\n2. HOLD AND
WAIT PREVENTION:");
        System.out.println("    - Processes
request all resources at beginning");
        System.out.println("    - Example:
Process requests {CPU, Printer, File} before
execution");

        System.out.println("\n3. NO
PREEMPTION:");
        System.out.println("    - Resources can
be forcibly taken from waiting processes");
        System.out.println("    - Example: OS
reclaims memory from stalled processes");

        System.out.println("\n4. CIRCULAR
WAIT PREVENTION:");
        System.out.println("    - Impose total
ordering on resources (R0 < R1 < R2)");
        System.out.println("    - Processes must
request resources in increasing order");

        // Interactive example
        System.out.println("\n--- INTERACTIVE
EXAMPLE ---");
        int numResources = getIntInput("Enter
number of resources for ordering example: ");

        System.out.println("Resource ordering:
");
        for (int i = 0; i < numResources; i++) {
            System.out.println("  R" + i + " < " + (i
< numResources - 1 ? "R" + (i + 1) : "END"));
        }

        System.out.println("\nEnter a resource
request sequence:");
        int prevResource = -1;
        boolean valid = true;

        for (int i = 0; i < numResources; i++) {
            int resource = getIntInput("  Request
resource R: ");
            if (resource <= prevResource) {
                System.out.println("  ✗ INVALID:
Resources must be requested in increasing
order!");
                valid = false;
                break;
            }
            prevResource = resource;
        }

        if (valid) {
            System.out.println("  ✓ VALID:
Resource request follows prevention rules");
        }

        System.out.println("\n5. PRACTICAL
IMPLEMENTATION STRATEGIES:");
        System.out.println("    - Timeout
mechanisms for resource requests");
        System.out.println("    - Process
termination when deadlock detected");
```

SOURCE CODE

```
        System.out.println("   - Resource preemption with rollback capability");
        System.out.println("   - Deadlock detection and recovery protocols");
    }
}
```

SCREEN SHOT/SAMPLE OUTPUT

1. Banker's Algorithm – Safe State Detection:

```
C:\Users\flexycode\.jdks\openjdk-22.0.2\bin\java.exe "-javaagent
==========================================
    DEADLOCK DETECTION & PREVENTION
          SYSTEM SIMULATION
==========================================


=== MAIN MENU ===
1. Banker's Algorithm - Safe State Detection
2. Banker's Algorithm - Resource Request
3. Resource Allocation Graph - Deadlock Detection
4. Deadlock Prevention Techniques
5. Exit
Choose an option (1-5): 1

=== BANKER'S ALGORITHM - SAFE STATE DETECTION ===
Enter number of processes: 5
Enter number of resource types: 3

--- AVAILABLE RESOURCES ---
Enter available instances for 3 resources:
  Resource R0: 3
  Resource R1: 3
  Resource R2: 2

--- PROCESS INFORMATION ---

Process P0:
Enter allocated resources for Process P0:
  Resource R0: 0
  Resource R1: 1
  Resource R2: 0
Enter maximum resources for Process P0:
  Resource R0: 7
  Resource R1: 5
  Resource R2: 3

Process P1:
Enter allocated resources for Process P1:
```

```
Process P1:
Enter allocated resources for Process P1:
   Resource R0: 2
   Resource R1: 0
   Resource R2: 0
Enter maximum resources for Process P1:
   Resource R0: 3
   Resource R1: 2
   Resource R2: 2

Process P2:
Enter allocated resources for Process P2:
   Resource R0: 3
   Resource R1: 0
   Resource R2: 2
Enter maximum resources for Process P2:
   Resource R0: 9
   Resource R1: 0
   Resource R2: 2

Process P3:
Enter allocated resources for Process P3:
   Resource R0: 2
   Resource R1: 1
   Resource R2: 1
Enter maximum resources for Process P3:
   Resource R0: 2
   Resource R1: 2
   Resource R2: 2

Process P4:
Enter allocated resources for Process P4:
   Resource R0: 0
   Resource R1: 0
   Resource R2: 2
Enter maximum resources for Process P4:
   Resource R0: 4
   Resource R1: 3
   Resource R2: 3
```

```
--- RUNNING BANKER'S ALGORITHM ---
  Executing P1, Work: [5, 3, 2]
  Executing P3, Work: [7, 4, 3]
  Executing P4, Work: [7, 4, 5]
  Executing P0, Work: [7, 5, 5]
  Executing P2, Work: [10, 5, 7]
  ✓ System is in SAFE state
  Safe sequence: P1 → P3 → P4 → P0 → P2

=== MAIN MENU ===
1. Banker's Algorithm - Safe State Detection
2. Banker's Algorithm - Resource Request
3. Resource Allocation Graph - Deadlock Detection
4. Deadlock Prevention Techniques
5. Exit
Choose an option (1-5):
```

2. Banker's Algorithm – Resource Request:

```
Choose an option (1-5): 2

=== BANKER'S ALGORITHM - RESOURCE REQUEST ===
Enter number of processes: 3
Enter number of resource types: 2

--- AVAILABLE RESOURCES ---
Enter available instances for 2 resources:
  Resource R0: 5
  Resource R1: 5

--- PROCESS INFORMATION ---

Process P0:
Enter allocated resources for Process P0:
  Resource R0: 2
  Resource R1: 1
Enter maximum resources for Process P0:
  Resource R0: 4
  Resource R1: 3

Process P1:
Enter allocated resources for Process P1:
  Resource R0: 1
  Resource R1: 2
Enter maximum resources for Process P1:
  Resource R0: 3
  Resource R1: 4

Process P2:
Enter allocated resources for Process P2:
  Resource R0: 1
  Resource R1: 1
Enter maximum resources for Process P2:
  Resource R0: 2
  Resource R1: 2
```

```
--- RESOURCE REQUEST ---
Enter process ID making request (0-2): 1
Enter resource request for Process P1:
  Resource R0: 1
  Resource R1: 0

--- PROCESSING REQUEST ---

  === Resource Request Simulation ===
  P1 requests: [1, 0]
  Checking if system remains safe after allocation...
  Executing P0, Work: [6, 6]
  Executing P1, Work: [8, 8]
  Executing P2, Work: [9, 9]
  ✓ System is in SAFE state
  Safe sequence: P0 → P1 → P2
  Request granted safely
```

3. Resource Allocation Graph – Deadlock Detection:

```
Choose an option (1-5): 3

=== RESOURCE ALLOCATION GRAPH ===
Enter number of processes: 3
Enter number of resources: 3
Enter instances for Resource R0: 1
Enter instances for Resource R1: 1
Enter instances for Resource R2: 1

--- REQUEST EDGES (Process → Resource) ---
Enter process-resource request pairs (enter -1 to stop):
  Process ID (-1 to stop): 0
  Resource ID: 0
  Process ID (-1 to stop): 1
  Resource ID: 1
  Process ID (-1 to stop): 2
  Resource ID: 2
  Process ID (-1 to stop): -1

--- ASSIGNMENT EDGES (Resource → Process) ---
Enter resource-process assignment pairs (enter -1 to stop):
  Resource ID (-1 to stop): 0
  Process ID: 1
  Resource ID (-1 to stop): 1
  Process ID: 2
  Resource ID (-1 to stop): 2
  Process ID: 0
  Resource ID (-1 to stop): -1
```

```
--- DETECTING DEADLOCK ---

  === Resource Allocation Graph ===
  Request Edges (Process → Resource):
    P0 → R0
    P1 → R1
    P2 → R2

  Assignment Edges (Resource → Process):
    R0 → P1
    R1 → P2
    R2 → P0

  × DEADLOCK DETECTED!
  Deadlock cycle: P0 → P1 → P2 → P0
```

4. Deadlock Prevention Techniques:

```
=== MAIN MENU ===
1. Banker's Algorithm - Safe State Detection
2. Banker's Algorithm - Resource Request
3. Resource Allocation Graph - Deadlock Detection
4. Deadlock Prevention Techniques
5. Exit
Choose an option (1-5): 4

=== DEADLOCK PREVENTION TECHNIQUES ===

1. MUTUAL EXCLUSION BREAKING:
   - Use shared resources or read-only access
   - Example: Multiple processes can read a file simultaneously

2. HOLD AND WAIT PREVENTION:
   - Processes request all resources at beginning
   - Example: Process requests {CPU, Printer, File} before execution

3. NO PREEMPTION:
   - Resources can be forcibly taken from waiting processes
   - Example: OS reclaims memory from stalled processes

4. CIRCULAR WAIT PREVENTION:
   - Impose total ordering on resources (R0 < R1 < R2)
   - Processes must request resources in increasing order

--- INTERACTIVE EXAMPLE ---
Enter number of resources for ordering example: 4
Resource ordering:
  R0 < R1
  R1 < R2
  R2 < R3
  R3 < END
```

```
Enter a resource request sequence:
  Request resource R: 0
  Request resource R: 2
  Request resource R: 3
  Request resource R: 4
  ✓ VALID: Resource request follows prevention rules

5. PRACTICAL IMPLEMENTATION STRATEGIES:
   - Timeout mechanisms for resource requests
   - Process termination when deadlock detected
   - Resource preemption with rollback capability
   - Deadlock detection and recovery protocols

=== MAIN MENU ===
1. Banker's Algorithm - Safe State Detection
2. Banker's Algorithm - Resource Request
3. Resource Allocation Graph - Deadlock Detection
4. Deadlock Prevention Techniques
5. Exit
Choose an option (1-5): 5
Thank you for using the Deadlock Detection System!

Process finished with exit code 0
```